Microsoft Graph Beta Copilot Admin API — Access, Permissions & Troubleshooting Runbook

Implementation Solution Guide

Version 1.0.0 · 2026-07-24

Contents

Executive Summary 5

Solution Overview 6

- The question this solves . . . 6
- What the "Graph Copilot admin" endpoint family is . . . 6
- The two things that make beta admin calls fail differently . . . 7
- GA vs beta — Microsoft's support posture (read before you build a dependency) . . . 8
- The diagnostic model (how the playbook is organized) . . . 8
- The standard deployment model . . . 9
- Benefits . . . 9
- Limitations and honest scope . . . 9
- Dedupe boundary — how this differs from the Agent Builder inventory product . . . 10

Requirements 10

- R1. Microsoft Graph tooling (PowerShell SDK or an equivalent HTTP client) . . . 10
- R2. A signed-in administrator (delegated user context) . . . 11
- R3. The correct permission scopes in the token . . . 11
- R4. A sufficient administrator role (Entra directory role) . . . 12
- R5. Admin consent granted for the scope in the tenant . . . 12
- R6. The correct cloud (sovereign-cloud endpoint) — and its availability limit . . . 12
- R7. Microsoft 365 Copilot licensing — and the Agent 365 note . . . 13
- R8. Tenant preview enablement (feature flighting) . . . 13
- R9. A named diagnostic evidence folder and output file . . . 14

Global Requirements — Copilot (category-level) 14

- G1. Microsoft 365 tenant with supported workloads . . . 14
- G2. Microsoft 365 Copilot licensing . . . 15
- G3. Microsoft Entra ID identity and MFA . . . 15
- G4. Data readiness — permissions & oversharing controls . . . 15
- G5. Information protection baseline (sensitivity labels) . . . 15
- G6. Administrative roles . . . 15
- G7. Network and client readiness . . . 15
- G8. Governance & responsible-use policy . . . 16

Diagnostic Runbook 16

- Step 0 — Establish the facts (always do this first) . . . 16
- Branch A — 401 Unauthorized (authentication failed) . . . 17
- Branch B — 403 with app-only token: "User context is required" . . . 17
- Branch C — 403 because the token is missing the scope (consent gap) . . . 18

Branch D — 403 because admin consent was never granted . . . 18
Branch E — 403 because the user lacks the admin role . . . 19
Branch F — 403/404 because the endpoint is not available in your cloud . . . 19
Branch G — 403 because a license prerequisite is not met (acknowledge, then exclude) . . . 20
Branch H — 404 because the path, namespace, or version is wrong . . . 20
Branch I — 429 Too Many Requests (throttling) . . . 21
Branch J — the data genuinely is not exposed (product limitation) . . . 21
After any fix — re-validate, then decide your dependency . . . 22
Testing & Validation 22
Gate T1 — The call authenticates and authorizes (200, not 4xx) . . . 23
Gate T2 — The response contains the data you actually need . . . 23
Gate T3 — The result is repeatable and correctly scoped to your cloud . . . 24
Gate T4 — A supported fallback returns the data (proves the exit path) . . . 24
Gate T5 — Evidence is captured . . . 24
Exit criteria . . . 25
Troubleshooting — edge cases and gotchas 25
Tooling, sign-in, and consent . . . 25
Throttling storms (429) . . . 26
Tenant-not-enabled (flighting) — the 403/404 that is not your fault . . . 26
Undocumented-endpoint deprecation and drift . . . 27
Policy-settings-specific responses (not failures) . . . 27
When the method itself stalls . . . 27
Best Practices — when to depend on beta vs wait for GA 28
1. Treat `/beta/copilot/admin/*` as interim, not production . . . 28
2. Prefer GA (`/v1.0`) or the admin center for anything you depend on . . . 28
3. Use delegated auth with a least-privilege admin role . . . 28
4. Request only the scopes you need, and name them exactly . . . 28
5. Wrap every beta dependency so it fails safe . . . 28
6. Pace your calls and honor throttling . . . 29
7. Plan for Government and sovereign clouds explicitly . . . 29
8. Re-verify on a schedule, and record what you excluded . . . 29
9. Keep the diagnostic evidence . . . 29
10. Don't reverse-engineer undocumented routes . . . 29
Deliverables Summary 29
Documents (Word + PDF) . . . 30
PowerShell scripts (5, read-only, cloud-aware) . . . 30

Named artifacts you create 31
What this solution deliberately does not include 31
How to use it 31
Appendix A — PowerShell Scripts 32

Executive Summary

The problem. Your team tried to call a Microsoft Graph **Copilot admin (beta)** endpoint — most often `GET /beta/copilot/admin/catalog/packages/{id}` to read the agent catalog — and the request was **blocked**. The error (a 401, 403, 404, or 429) looks the same for a dozen unrelated causes, and the preview documentation does not reliably tell them apart. Time is lost guessing between a missing permission, a missing admin role, an app-vs-user token problem, an ungranted consent, a preview that has not reached your tenant, a cloud where the feature does not exist, a license prerequisite, a wrong path, or a capability that simply is not there yet.

The solution. This is a **branch-based troubleshooting runbook** for the entire `/beta/copilot/admin/*` surface. It starts with a Step-0 fact-finding pass, then routes your exact HTTP status and error text to **one named root cause** (Branches A–J), each with how to *confirm* it, the *fix or supported workaround*, and how to *re-validate*. Five read-only PowerShell scripts do the diagnosis for you: they report your token scopes and admin roles, probe the endpoint and decode the real error, resolve *have-vs-need*, try the supported fallbacks, and write a single findings file you can attach to a Microsoft support case.

Why it is trustworthy. Every endpoint path, permission scope, admin role, error meaning, license requirement, and cloud-availability fact was **verified against Microsoft Learn on the build date** — the Package Management API reference, the policy-settings reference, the Copilot APIs overview and security pages, and the Graph permissions, throttling, and national-cloud references. Because this is a **preview** surface that changes without notice, every volatile claim is explicitly flagged *"verify against current Microsoft documentation."* The runbook never invents an endpoint, scope, role, or response shape — where the honest answer is *"the data is not exposed"* or *"do not depend on this,"* it says so and points you to the supported path.

Key facts your team will act on immediately:

- These admin endpoints require a **signed-in user (delegated)**; **application-only tokens are rejected** with *"User context is required"* — a leading cause of the 403 people cannot explain.
- **Scope and admin role are separate gates.** A token with `CopilotPackages.Read.All` but a user without **AI Administrator / Global Administrator** still gets 403.
- The endpoints are **Global-only** today — **not available in GCC High, DoD, or 21Vianet** — so in a Government tenant the fallback is the primary path.
- Microsoft's current docs state the package catalog now **requires a Microsoft Agent 365 license** (a change from the original announcement). **Your team has already ruled this out** as the blocker; the runbook confirms and excludes it, then moves to the real cause.
- **Beta is "not supported for production."** The runbook's decision guidance keeps you on GA (`/v1.0`), admin-center export, or Purview audit for anything you depend on.

The outcome. A blocked Graph Copilot admin call turned into a single named cause and fixed — or, where the beta surface cannot be relied on, a proven, supported alternative delivering the data you need. A repeatable method and scripts your team keeps and reuses on the next preview-API investigation.

Solution Overview

The question this solves

You tried to call a **Microsoft Graph Copilot admin (beta) endpoint** — most commonly `GET https://graph.microsoft.com/beta/copilot/admin/catalog/packages` or `GET https://graph.microsoft.com/beta/copilot/admin/catalog/packages/{id}` — and the request was **blocked or failed**. You got back a `401`, a `403`, a `404`, or a `429`, and you cannot tell which of several very different things is actually wrong:

- your token is missing the right **permission scope**, or
- your signed-in user is missing the right **admin role**, or
- you used an **application-only token** where the endpoint demands a **signed-in user**, or
- **admin consent** was never granted in the tenant, or
- the **beta/preview feature has not rolled out** to your tenant yet, or
- the endpoint **is not available in your cloud** (US Gov GCC High / DoD / 21Vianet), or
- a **license prerequisite** applies, or
- the **path/namespace/version is wrong** or has been renamed, or
- you are being **throttled**, or
- the data you want **simply is not exposed** by the beta endpoint yet.

Every one of those returns a similar-looking error, and the beta docs do not always tell them apart. This runbook is a **branch-based diagnostic method** that turns the raw HTTP status into a single, named root cause — then hands you the exact fix or the supported fallback for each. It is deliberately **general to the whole `/beta/copilot/admin/*` surface**, not to one report. It is the deep troubleshooting companion to the inventory/export product described under **Dedupe boundary** below.

> *This is an **API troubleshooting runbook**, not a feature tutorial. Its job is to get a *blocked or failing* Graph Copilot admin request working, prove whether the data you need is actually exposed, and — where the beta endpoint cannot be relied on — route you to the **supported** alternative.*

What the "Graph Copilot admin" endpoint family is

Microsoft 365 Copilot exposes a set of **admin** operations under the Microsoft Graph `copilot` namespace. They are available under two versions:

- `https://graph.microsoft.com/v1.0/copilot/...` — the **GA (generally available)** surface.
- `https://graph.microsoft.com/beta/copilot/...` — the **beta/preview** surface. Microsoft's

standing notice on every beta page: *"APIs under the `/beta` version are subject to change. Use of these APIs in production applications is not supported."*

Within that namespace, the **admin** operations this runbook targets are:

Operation family	Example endpoint	Purpose
Package Management (agent catalog)	`GET /beta/copilot/admin/catalog/packages` and `.../packages/{id}`	List agents/apps in the org catalog and read one package's detail (metadata + elements). Also GA at `/v1.0/copilot/admin/catalog/packages`.
Policy settings	`GET /beta/copilot/admin/policySettings/{id}`	Read a tenant-level Copilot policy setting value.
Usage reports	`GET /v1.0/copilot/reports/getMicrosoft365CopilotUsageUserDetail`	GA usage/adoption data (a supported fallback when a beta admin call is blocked).

`GET /beta/copilot/admin/catalog/packages/{id}` — the endpoint most customers hit first — returns a `copilotPackageDetail` object: the agent's `id`, `displayName`, `type` (`custom` / `external` / etc.), `publisher`, `isBlocked`, `supportedHosts`, `availableTo` / `deployedTo`, `elementTypes`, and per-element `elementDetails`. The list endpoint returns a collection of the lighter `copilotPackage` objects and supports `$filter` on `supportedHosts`, `elementTypes`, `lastModifiedDateTime`, and `platform`.

> ***Accuracy note (beta is volatile).*** *Every endpoint path, scope name, license requirement, admin role, and error meaning in this runbook was verified against Microsoft Learn on the build date (see the Requirements guide for exact source pages). Because this is a **preview** surface, Microsoft can change any of it without notice. Where current behavior is known to diverge from the published reference, this guide says so explicitly and marks it **"verify against current Microsoft documentation — beta, subject to change."** Do not treat any single response shape as permanent.*

The two things that make beta admin calls fail differently

Two distinctions decide almost every failure you will see. Learn them first; the whole playbook turns on them.

1. Authentication vs authorization (401 vs 403)

• A `401 Unauthorized` is an **authentication** failure: Graph could not validate *who is

calling*. The token is missing, expired, malformed, issued for the wrong **resource/audience**, or issued in the **wrong cloud** (a commercial token sent to `graph.microsoft.us`, or vice-versa).

• A `403 Forbidden` is an **authorization** failure: Graph knows who you are but you are **not

allowed **to do this. That can be a missing** scope**, a missing** admin role**, an** app-only token where a user is required**, an ungranted** admin consent**, a** feature not enabled for the tenant**, or a** license** gap. A 403 is therefore *ambiguous by itself* — the runbook's core job is to disambiguate it.

2. Scope (consent) vs role (directory role)

A Graph call to a tenant-wide admin resource needs **both**:

• **A permission scope** — e.g. `CopilotPackages.Read.All` — present in the **token**. A scope is

granted by **consent** (admin consent, for `.All` application-style scopes). A missing scope is a *consent/registration* problem.

- **A sufficient directory role** — for delegated calls, the **signed-in user** must hold an admin

role that authorizes reading the tenant-wide agent catalog (see Requirements). A user with the right scope in their token but **no admin role** is still refused. A missing role is an *assignment* problem, fixed in Entra, not in the app registration.

Missing-scope and missing-role can both surface as `403`, which is exactly why people "have the permission" and still get denied. The playbook separates them with a preflight that reports **scopes in the token** and **roles on the user** side by side.

GA vs beta — Microsoft's support posture (read before you build a dependency)

	GA (`/v1.0`)	Beta (`/beta`)
Support statement	Supported for production use.	*"Not supported"* for production; *"subject to change."*
Breaking changes	Follow Graph's deprecation/versioning policy (advance notice).	Can change **shape, path, permissions, or availability without notice**.
Throttling	Standard Graph limits, `Retry-After` honored.	Similar limits but preview services often have **tighter, less predictable** burst capacity.
Tenant availability	Broadly available.	Frequently **rolling out tenant-by-tenant**; a correct call can still 403/404 until your tenant is flighted.
Cloud availability	Varies per API; check the per-endpoint national-cloud table.	Copilot admin beta endpoints are **Global-only** today (not GCC High / DoD / 21Vianet).

Decision rule this runbook recommends: treat a `/beta/copilot/admin/*` endpoint as a **diagnostic and interim** data source, not a production dependency. If a GA (`/v1.0`) equivalent exists (e.g. the GA `catalog/packages` path, or the GA usage reports), prefer it. If only beta exists and you must depend on it, **wrap it** so a shape/permission change or a 404 fails safe, and re-check against the current docs on a schedule. Best-practices guidance is in `07-best-practices.md`.

The diagnostic model (how the playbook is organized)

The runbook is a **symptom** → **cause** → **fix** → **re-validate** tree, in a fixed order:

1. **Step 0 — Establish the facts.** Capture the *exact* request (method, full URL, version, cloud),

the *exact* status code, and the *exact* error `code` / `message` / `request-id`. Read what scopes are in your token and what roles are on your user. You cannot branch correctly without this.

1. **Branches A–J.** Each branch owns one root cause (insufficient scope, missing role, app-only vs

delegated, consent not granted, preview not rolled out, cloud not enabled, license prerequisite, wrong path/namespace, throttling, genuine product limitation). Each gives you the **symptom**, how to **confirm** it, the **fix or supported workaround**, and how to **re-validate**.

1. **Re-validate** with the testing procedures in `05-testing.md`, which also prove the harder

question: whether the data you need **is or is not** exposed by the endpoint at all.

The standard deployment model

Every solution in this catalog follows the same shape so each engagement feels familiar and low-risk: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.** Here that means: assess the exact failing call (Step 0); configure the missing scope/role/consent/cloud target; pilot the fixed call with a single delegated admin; validate against the testing gates; then expand to your automation (scheduled export, report, or integration) — or, if the beta endpoint proves unreliable, expand onto the **supported fallback** instead.

Benefits

• **Turns an ambiguous 403 into one named cause** — no more guessing between scope, role, consent,

rollout, cloud, and license.

• **Verified against Microsoft Learn**, with every beta-volatile claim flagged so you know what to

re-check.

• **Read-only and safe** — the five scripts diagnose and report; none of them change tenant state.

• **Cloud-aware** — every script targets Global, GCC High, or DoD Graph via a single switch.

• **Gives you the supported exit** — when beta cannot be relied on, you leave with the GA endpoint,

admin-center export, or audit-log path that *can*.

Limitations and honest scope

• This runbook covers **diagnosing and unblocking requests to the Graph Copilot admin (beta) surface

and choosing a supported alternative.** It is not a guide to *building* agents, and it does not and cannot make Microsoft expose data the API does not expose.

• Where a beta endpoint is **undocumented or not intended for third-party app use**, the honest answer

is to **not depend on it** and use the supported path; the playbook says so rather than inventing a workaround.

• The customer has already **ruled out Agent 365 licensing** as their blocker; this guide acknowledges

the license prerequisite that Microsoft's current docs state (Branch G) and then moves past it to the other causes.

Dedupe boundary — how this differs from the Agent Builder inventory product

A sibling product, **Copilot Agent Builder — Inventory, Sharing, Governance & Export**, delivers the actual **agent inventory + sharing export** deliverable (a scheduled, filtered register of Agent Builder agents and who they are shared with) and includes a short diagnostic for that one task. **This product is different:** it is the **deep, general troubleshooting runbook for the** `/beta/copilot/admin/*` **API surface itself** — every failure cause, across every admin operation in that namespace, with the scripts to confirm each. If your goal is *the export*, use the inventory product. If your goal is *understanding and fixing why any Graph Copilot admin call is blocked*, this is the runbook. Where you need the export after unblocking the call, this guide points you to that product rather than reproducing it.

Requirements

These are the prerequisites to **call** the Graph Copilot admin (beta) endpoints and to run this runbook's diagnostics. Confirm each before you start; missing any one of them is itself one of the failure causes the playbook diagnoses. Every entry follows **Description / Why / Verify / Implement-if-missing.**

> ***Sources verified on the build date*** *(all* `learn.microsoft.com`*): the Package Management API overview and its* `copilotpackages-list` */* `copilotpackagedetail-get` *reference pages; the* `copilotpolicysetting-get` *reference; the *Microsoft 365 Copilot APIs overview* and *Security and authentication for Microsoft 365 Copilot APIs* concept pages; the Microsoft Graph *permissions reference*, *throttling guidance*, and *national cloud deployments* pages. Because this is a beta surface, re-verify anything marked* ***"beta — verify against current Microsoft documentation."***

R1. Microsoft Graph tooling (PowerShell SDK or an equivalent HTTP client)

Description: The **Microsoft Graph PowerShell SDK** (`Microsoft.Graph` and, for beta paths, the `Microsoft.Graph.Beta` modules), or any HTTP client (Graph Explorer, `curl`, Postman) that can send a bearer token to `https://graph.microsoft.com`. This runbook's scripts use `Connect-MgGraph` and `Invoke-MgGraphRequest`.

Why: The Copilot admin endpoints are ordinary Microsoft Graph REST endpoints; you reach them with the same tooling and token flow as any other Graph call. `Invoke-MgGraphRequest` lets you call a raw beta path (including one with no typed SDK cmdlet) and read the **real** status code and error body — which is exactly what diagnosis needs.

Verify: `Get-Module Microsoft.Graph -ListAvailable` returns a version; `Get-Command Connect-MgGraph` resolves.

Implement if missing: `Install-Module Microsoft.Graph -Scope CurrentUser` and `Install-Module Microsoft.Graph.Beta -Scope CurrentUser`. Update an old SDK with `Update-Module Microsoft.Graph`. (Graph Explorer at `https://developer.microsoft.com/graph/graph-explorer` is a good zero-install way to reproduce a call interactively.)

R2. A signed-in administrator (delegated user context)

Description: A **work or school account** you can sign in with interactively — **not** an application-only (client-credentials) identity — that holds a sufficient **admin role** (R4).

Why: The Copilot admin endpoints authenticate and authorize the **signed-in user**. In current behavior, the package catalog endpoint **rejects application-only tokens** with `403 Forbidden` / *"Access is denied. User context is required by the requested resource."* even when the app has the `CopilotPackages.Read.All` **application** permission granted — despite the reference page listing Application permissions as supported. The policy-settings endpoint documents **Application as "Not supported"** outright. **Treat these endpoints as delegated-only.** *(Beta — verify against current Microsoft documentation; the documented-vs-enforced permission model has been changing.)*

Verify: After `Connect-MgGraph`, run `(Get-MgContext).Account` — it must show a **user** UPN, not a service-principal/app id. `(Get-MgContext).AuthType` should be `Delegated`.

Implement if missing: Sign in interactively: `Connect-MgGraph -Scopes "CopilotPackages.Read.All" -Environment Global` (or device-code: `Connect-MgGraph -UseDeviceAuthentication`). Do not use `-ClientSecret` / certificate app-only auth for these endpoints.

R3. The correct permission scopes in the token

Description: The token you present must **contain** the scope the endpoint requires:

Endpoint	Least-privileged delegated scope	Higher-privileged	App permission
`catalog/packages` (list) and `catalog/packages/{id}` (read)	`CopilotPackages.Read.All`	`CopilotPackages.ReadWrite.All`	Documented but **not reliably enforced** — treat as delegated-only (R2)
`catalog/packages/{id}` block/unblock/reassign/update (write, preview)	`CopilotPackages.ReadWrite.All`	—	Same caveat
`policySettings/{id}` (read)	`CopilotPolicySettings.Read`	`CopilotPolicySettings.ReadWrite`	**Not supported**
`reports/getMicrosoft365CopilotUsageUserDetail` (GA fallback)	`Reports.Read.All`	—	`Reports.Read.All`

Why: A token missing the scope yields `403 Forbidden`. The scope names are **exact** — do not guess or abbreviate them. Note that `CopilotPackages.*` and `CopilotPolicySettings.*` are **different** scope families for **different** endpoints; a token good for one is not good for the other.

Verify: After connecting, `(Get-MgContext).Scopes` lists the scopes actually in your token. The runbook's `CPADMDX-Resolve-Scopes.ps1` reports **have vs need** for the endpoint you name. (You can also paste the access token into `https://jwt.ms` and read the `scp` claim — never paste a token

anywhere else.)

Implement if missing: Reconnect requesting the scope, e.g. `Connect-MgGraph -Scopes "CopilotPackages.Read.All","CopilotPolicySettings.Read"`. If the tenant blocks user consent, you also need R5 (admin consent).

R4. A sufficient administrator role (Entra directory role)

Description: The signed-in user must hold an admin role that authorizes tenant-wide Copilot/agent administration. The **least-privileged** built-in role for Microsoft 365 Copilot and Microsoft Agent 365 administration is **AI Administrator**; **Global Administrator** is the superset that always satisfies it.

Why: Even with the right scope in the token (R3), a **delegated** call to a tenant-wide admin resource is refused if the *user* lacks an admin role — a `403 Forbidden`. Scope (consent) and role (assignment) are **independent** gates; you need both. *(Microsoft's Graph reference pages for the package endpoints do not name an exact directory role; **AI Administrator / Global Administrator** is the documented role for managing agents in the Microsoft 365 admin center. **Beta — verify the exact role the Graph endpoint honors against current Microsoft documentation.**)*

Verify: Entra admin center → **Roles and administrators** → search the user; or the runbook's `CPADMDX-Preflight.ps1` reports the user's directory-role memberships (via `GET /me/memberOf/microsoft.graph.directoryRole`).

Implement if missing: Assign **AI Administrator** (preferred, least privilege) to the operator in Entra → **Roles and administrators** → *AI Administrator* → **Add assignments**; reserve **Global Administrator** for people who genuinely need tenant-wide control. Allow for role-activation delay if the role is PIM-eligible rather than active.

R5. Admin consent granted for the scope in the tenant

Description: The `CopilotPackages.*` and `CopilotPolicySettings.* .All`/admin scopes require **admin consent** in the tenant for the app you sign in through (for interactive SDK use, that app is **Microsoft Graph Command Line Tools**, appId `14d82eec-204b-4c2f-b7e8-296a70dad67c`).

Why: If the tenant disables user consent and no admin has consented, the sign-in either cannot grant the scope (so R3 fails) or Graph returns `403`. Consent is how a scope actually lands in a token.

Verify: Entra → **Enterprise applications** → *Microsoft Graph Command Line Tools* → **Permissions** shows the Copilot scopes as **granted for the organization**; or attempt `Connect-MgGraph -Scopes ...` and read whether consent was blocked.

Implement if missing: A Global Administrator grants admin consent once — either at the interactive consent prompt (**Consent on behalf of your organization**) or in Entra → the app → **Grant admin consent**. For a custom app registration, grant the delegated Copilot scopes on its **API permissions** blade and click **Grant admin consent**.

R6. The correct cloud (sovereign-cloud endpoint) — and its availability limit

Description: Target the Graph endpoint for **your** cloud:

Cloud	Graph host	`Connect-MgGraph -Environment`	Copilot admin beta available?
Commercial / Global	`graph.microsoft.com`	`Global`	**Yes**
US Gov GCC High	`graph.microsoft.us`	`USGov`	**No** (Global-only today)
US Gov DoD	`dod-graph.microsoft.us`	`USGovDoD`	**No** (Global-only today)
China (21Vianet)	`microsoftgraph.chinacloudapi.cn`	`China`	**No**

Why: A token from one cloud sent to another cloud's host fails authentication (`401`). More importantly, the **Package Management** and **policy-settings** admin endpoints are published as **Global-only** — their national-cloud table shows ■ for US Gov L4, US Gov L5 (DoD), and 21Vianet. In a Gov tenant the call can therefore fail even when scope, role, consent, and cloud target are all correct, because the **feature is not offered in that cloud** (Branch F). *(Beta — cloud availability can change; verify the per-endpoint national-cloud table for GCC High/DoD against current Microsoft documentation.)*

Verify: `(Get-MgContext).Environment` shows the cloud you connected to. The scripts print the resolved Graph host so you can confirm you hit the intended cloud.

Implement if missing: Connect with the matching switch, e.g. `Connect-MgGraph -Environment USGov -Scopes "CopilotPackages.Read.All"`. If you are in GCC High/DoD and the endpoint is unavailable there, use the supported fallbacks (`04`-testing and Branch F): admin-center export, GA endpoints that *are* offered in your cloud, and Purview audit logs.

R7. Microsoft 365 Copilot licensing — and the Agent 365 note

Description: A **Microsoft 365 Copilot** license on a qualifying base plan (E3/E5 or equivalent) for the users whose Copilot data you work with. Separately, Microsoft's **current** Package Management API docs state: *"Access to the Package Management API requires a Microsoft Agent 365 license."*

Why: Copilot API capabilities are gated on Copilot licensing. The **Agent 365** requirement for the package catalog endpoint is **newer and changed**: the original announcement said *"no additional license or add-on is required,"* and the reference pages were later updated to require **Agent 365**. This is exactly the kind of preview-volatile prerequisite to check. **The customer has already ruled out Agent 365 licensing as their blocker** — so treat this requirement as *acknowledged and excluded*, and use Branch G only to *confirm* it is not the cause before moving on.

Verify: Microsoft 365 admin center → **Billing** → **Licenses** shows Copilot (and, if applicable, Agent 365) seats assigned to the relevant users.

Implement if missing: Assign the appropriate licenses (out of scope for this runbook to purchase). If Agent 365 is genuinely required by your target endpoint and you do not have it, that is a **product limitation for you today** (Branch J), not a code bug — pursue the supported fallback. *(Beta — verify the current license requirement on the endpoint's reference page.)*

R8. Tenant preview enablement (feature flighting)

Description: The beta Copilot admin endpoints roll out **tenant-by-tenant**. Your tenant must be **flighted** for the specific endpoint.

Why: A correctly-formed, correctly-authorized call can still return `403 Forbidden` / *"Access is denied to the requested resource."* or `404` purely because the preview feature has not reached your tenant. Multiple tenants report this on Microsoft Q&A for `catalog/packages`. There is no self-service switch — it is Microsoft-side flighting.

Verify: Compare the same call in a tenant known to be enabled, or confirm via the Microsoft 365 admin center whether the **Agents** / Agent Registry surface (which the API mirrors) is present. If the UI surface is absent, the API is likely not flighted either.

Implement if missing: Wait for the rollout, raise a Microsoft support case for eligibility, and in the meantime use the supported fallbacks (Branch F/J). Do not keep retrying a flighting gap in a tight loop — it will not change until Microsoft enables it.

R9. A named diagnostic evidence folder and output file

Description: A working folder for the diagnostic outputs this runbook produces. Name it exactly `CPADMDX-Diagnostics`, and have the scripts write their summary to `CPADMDX-Findings.json` (and a human-readable `CPADMDX-Findings.txt`) inside it.

Why: Diagnosis is only useful if it is captured. Reusing one fixed folder and file name keeps evidence together across preflight, probe, scope-resolution, and fallback tests, and gives you a single artifact to attach to a Microsoft support case. Every script and every testing gate in this solution refers to `CPADMDX-Diagnostics\CPADMDX-Findings.json` by that exact name.

Verify: The folder exists and `CPADMDX-Findings.json` is written after a run.

Implement if missing: `New-Item -ItemType Directory -Path .\CPADMDX-Diagnostics` (the `CPADMDX-Report-Findings.ps1` script creates it if absent).

See `global-requirements-copilot.md` for the category-wide Copilot prerequisites (tenant, licensing, identity, data readiness, roles, governance) that apply to every Copilot solution.

Global Requirements — Copilot (category-level)

These requirements apply to every solution in the Copilot category. They are published once at the category level so individual listings stay small. Confirm these before deploying any Microsoft 365 Copilot solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Copilot inherits the tenant's existing identity, permission, and data-governance posture. It creates no new access — but it surfaces everything a user already has. Treat data readiness (oversharing, labeling) as a prerequisite, not an afterthought, and pilot before broad enablement.*

G1. Microsoft 365 tenant with supported workloads

Description: A Microsoft 365 tenant with Exchange Online, SharePoint Online, OneDrive, and Teams in use. Why: Copilot reasons over Microsoft Graph content across these workloads; they must be provisioned and healthy. Verify: Microsoft 365 admin center shows the services active; users have

mailboxes and OneDrive provisioned. Implement: Provision/enable the M365 workloads and confirm users are licensed for them before adding Copilot.

G2. Microsoft 365 Copilot licensing

Description: Microsoft 365 Copilot licenses for the target users (a prerequisite base M365 plan applies). Why: Copilot is a per-user add-on; a user cannot use Copilot without an assigned license on a qualifying base plan. Verify: Admin center -> `Billing -> Licenses`; confirm Copilot licenses and free seats. *Verify current base-plan prerequisites in your tenant.* Implement: Purchase/assign Microsoft 365 Copilot licenses to the pilot cohort; assign via group where possible.

G3. Microsoft Entra ID identity and MFA

Description: Cloud identities in Entra ID with MFA and healthy sign-in. Why: Copilot honors identity and Conditional Access; a weak or broken identity plane undermines both access and security. Verify: Users sign in at `https://entra.microsoft.com` resources; MFA is enforced for the cohort. Implement: Ensure licensed, MFA-protected Entra accounts (see the Entra category solutions for identity hardening).

G4. Data readiness — permissions & oversharing controls

Description: A known state of SharePoint/OneDrive permissions and sharing (no unmanaged broad access). Why: Copilot can surface any content the user has access to; oversharing becomes AI data exposure. Verify: Review sharing reports / SharePoint Advanced Management signals; run the solution's discovery script. *SAM/Restricted Content Discovery availability varies — verify in your tenant.* Implement: Remediate broad-access sites and links before broad Copilot enablement (see the oversharing-remediation solution).

G5. Information protection baseline (sensitivity labels)

Description: A published sensitivity-label taxonomy (even minimal) in Microsoft Purview. Why: Labels are how you keep sensitive content out of AI processing where required and audit what Copilot uses. Verify: Purview compliance portal -> Information Protection shows published labels. Implement: Publish a baseline label set (see the Purview information-protection solutions) before enforcing AI guardrails.

G6. Administrative roles

Description: Global Administrator (or scoped admin roles: SharePoint Admin, Security/Compliance Admin) for the operator. Why: Enabling Copilot, remediating SharePoint, and configuring Purview DLP/labels require these roles. Verify: Entra -> `Roles and administrators`; confirm the required roles. Implement: Assign least-privilege roles for the task; reserve Global Administrator for tenant-wide changes.

G7. Network and client readiness

Description: Supported Microsoft 365 Apps clients and network access to Microsoft 365/Copilot endpoints. Why: Copilot experiences in Word/Excel/Outlook/Teams require current clients and connectivity. Verify: Clients on a supported channel/build; Microsoft 365 endpoints reachable. *Verify current client requirements.* Implement: Update the M365 Apps channel/build and allow the required endpoints.

G8. Governance & responsible-use policy

Description: An organizational position on acceptable AI use, data handling, and auditability. Why: Copilot generates and surfaces content; the organization must define what is acceptable and keep an audit trail. Verify: A documented AI-use/acceptable-use policy exists; Copilot interactions are auditable (Purview audit). Implement: Adopt an AI-use policy; enable audit/eDiscovery for Copilot. *Governance and acceptable-use determinations are the customer's compliance/legal decision — this guide covers the technical controls only.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

Diagnostic Runbook

This is the core of the solution: a **branch-based diagnostic tree** for a blocked or failing request to a `/beta/copilot/admin/*` (or `/v1.0/copilot/admin/*`) endpoint. Start from what you observe in **Step 0**, follow the branch that matches your exact status code and error text, run the checks, apply the fix or supported workaround, then re-validate with the gates in `05-testing.md`.

Golden rule: the request passes through an ordered set of gates — **Authentication** → **Consent (scope)** → **Admin role** → **Delegated-vs-app** → **Tenant/cloud availability** → **License** → **Path/version** → **Throttle**. The **first** gate that fails is the root cause; everything after it is a symptom. Fix the first failing gate, then re-run the probe. Never invent a response shape for a beta endpoint — if the call fails, read the **real** status and error body and branch on it.

Step 0 — Establish the facts (always do this first)

You cannot branch correctly without the exact request and the exact error. Capture all of it before you change anything.

1. **Record the exact request.** Write down, verbatim: the **HTTP method**, the **full URL** including

the **version segment** (`/v1.0/` vs `/beta/`), any `$filter`, and the **cloud host** you sent it to (`graph.microsoft.com`, `graph.microsoft.us`, or `dod-graph.microsoft.us`). A wrong version or wrong host is itself a cause (Branches F and H).

1. **Reproduce it with the real status visible.** Run `CPADMDX-Probe-Endpoint.ps1` against the same

endpoint. It sends the request with `Invoke-MgGraphRequest`, and on failure it prints the **HTTP status code**, the error `code`, the error `message`, and the `request-id` / `client-request-id` from the body. Do not rely on the console UI's paraphrase — read the raw response. Example of what you are capturing:

```
{
  "error": {
    "code": "Forbidden",
    "message": "Access is denied to the requested resource.",
    "innerError": {
      "request-id": "00000000-0000-0000-0000-000000000000",
      "client-request-id": "00000000-0000-0000-0000-000000000000",
      "date": "2026-07-24T00:00:00"
    }
  }
```

```
}
```

1. **Read your identity, scopes, and roles.** Run `CPADMDX-Preflight.ps1`. It reports, in one

place: the **signed-in account** and **auth type** (delegated vs app — R2), the **scopes in the token** (R3), the user's **directory roles** (R4), and the **cloud** you connected to (R6). This is the single most important input to branching — most "I have the permission but still get 403" cases are a **role** gap or an **app-vs-delegated** gap, both visible here.

1. **Write the facts to evidence.** `CPADMDX-Report-Findings.ps1` records everything into

`CPADMDX-Diagnostics\CPADMDX-Findings.json`. Keep it; it is your support-case attachment.

Now pick the branch whose **symptom** matches. If two could apply, take them **in order A → J** (that is gate order).

Branch A — 401 Unauthorized (authentication failed)

Symptom: `401 Unauthorized`. Error `code` is typically `InvalidAuthenticationToken` with a message like *"Access token is empty / expired / has invalid signature"*, or *"CompactToken parsing failed"*, or an audience/issuer mismatch.

Owner gate: Authentication — Graph could not validate *who* is calling.

Confirm:

1. In `CPADMDX-Preflight.ps1` output, check whether you are even connected: `Get-MgContext` returns

`$null` if no token. A `401` with no context means you never authenticated.

1. Check **token age** — access tokens expire (typically ~60–90 min). A long-running script that

cached a token can 401 midway. Reconnect and retry.

1. Check the **audience/resource and cloud**. The token must be issued for

`https://graph.microsoft.com` (Global) or the matching Gov host. A token minted for `graph.microsoft.us` sent to `graph.microsoft.com` (or vice-versa) 401s. Confirm `(Get-MgContext).Environment` matches the host in your URL.

Fix / workaround: Reconnect with a fresh interactive sign-in in the **correct cloud**: `Connect-MgGraph -Environment Global -Scopes "CopilotPackages.Read.All"` (or `USGov` / `USGovDoD`). Ensure the client acquiring the token targets the Graph resource, not another API.

Re-validate: Re-run `CPADMDX-Probe-Endpoint.ps1`. A `401` should not recur; you will now get either `200` (done) or a `403`/`404` — branch again on the new status.

Branch B — 403 with app-only token: "User context is required"

Symptom: `403 Forbidden` and the message is specifically *"Access is denied. User context is required by the requested resource."* You are calling with **application (client-credentials)** authentication.

Owner gate: Delegated-vs-app — the endpoint demands a **signed-in user**; an app-only token is refused *regardless of granted application permissions*.

Confirm:

1. `CPADMDX-Preflight.ps1` shows **AuthType = AppOnly** (or `Get-MgContext` shows a service-principal

`ClientId` and no user `Account`).

1. Your app registration has `CopilotPackages.Read.All` as an **Application** permission with admin

consent — and it *still* 403s with the "user context is required" message. That combination is the signature of this branch. It matches Microsoft's own reference stating the policy-settings endpoint does not support Application permissions, and community confirmation that the packages endpoint enforces delegated context despite the reference listing Application permissions.

Fix / workaround: Switch to **delegated** authentication with a user who holds the admin role (R4): `Connect-MgGraph -Scopes "CopilotPackages.Read.All" -Environment Global` (interactive) or `Connect-MgGraph -UseDeviceAuthentication`. If you need unattended automation, this is a genuine constraint of the beta endpoint — see Branch J and `07-best-practices.md` for the supported unattended paths (GA usage reports with `Reports.Read.All` application permission; admin-center scheduled export).

Re-validate: `(Get-MgContext).AuthType` now reads `Delegated`; re-run the probe. *(Beta — verify against current Microsoft documentation; if Microsoft later enables application permissions on this endpoint, this branch no longer applies.)*

Branch C — 403 because the token is missing the scope (consent gap)

Symptom: `403 Forbidden` (`code: Forbidden`, *"Access is denied to the requested resource."*) with a **delegated** token, and the required scope is **not in the token**.

Owner gate: Consent (scope).

Confirm:

1. Run `CPADMDX-Resolve-Scopes.ps1` and name your endpoint. It prints **have vs need**. If, for

`catalog/packages`, `CopilotPackages.Read.All` is in the **need** column but **absent** from the **have** column, this is your branch.

1. Cross-check `(Get-MgContext).Scopes`. Remember the families differ: `policySettings` needs

`CopilotPolicySettings.Read`, **not** `CopilotPackages.Read.All`.

Fix / workaround: Reconnect requesting the exact scope: `Connect-MgGraph -Scopes "CopilotPackages.Read.All"` (or `"CopilotPolicySettings.Read"`). If the consent prompt is blocked, you have a tenant consent-policy gap → get admin consent (Branch D).

Re-validate: `CPADMDX-Resolve-Scopes.ps1` shows the scope now present; re-run the probe.

Branch D — 403 because admin consent was never granted

Symptom: You **request** the correct scope at sign-in but it is **not granted** — either the consent prompt says an administrator must approve, or the scope silently never lands in the token, and the call 403s. The scope family requires **admin consent**.

Owner gate: Consent (tenant policy) — user consent is insufficient/blocked for an admin scope.

Confirm:

1. `CPADMDX-Resolve-Scopes.ps1` shows the scope in **need** but it never appears in **have** even

though you asked for it at `Connect-MgGraph`.

1. Entra → **Enterprise applications** → *Microsoft Graph Command Line Tools*

(appId `14d82eec-204b-4c2f-b7e8-296a70dad67c`) → **Permissions** does **not** list the Copilot scope as granted for the organization.

Fix / workaround: A **Global Administrator** grants admin consent **once**: at the interactive prompt choose *"Consent on behalf of your organization,"* or in Entra → the app → **Grant admin consent for** `<tenant>`. For a custom app registration, add the delegated Copilot scope on **API permissions** and click **Grant admin consent**.

Re-validate: Reconnect; `CPADMDX-Resolve-Scopes.ps1` now shows the scope present; re-run the probe.

Branch E — 403 because the user lacks the admin role

Symptom: `403 Forbidden` with a **delegated** token that **does** contain the correct scope (C and D are clean) — but the request is still denied. This is the classic *"I have the permission and still get 403."*

Owner gate: Admin role — scope present, but the **signed-in user** is not an admin for this resource.

Confirm:

1. `CPADMDX-Preflight.ps1` shows the scope **present** in the token **and** the user's directory-role

list contains **neither AI Administrator nor Global Administrator** (nor another role Microsoft documents for this surface).

1. `CPADMDX-Resolve-Scopes.ps1` confirms the scope is satisfied — isolating the failure to **role**,

not scope.

Fix / workaround: Assign **AI Administrator** (least privilege) to the user in Entra → **Roles and administrators** → *AI Administrator* → **Add assignments** (or use an account that already holds it; Global Administrator also works). If the role is **PIM-eligible**, **activate** it and wait for activation before retrying.

Re-validate: `CPADMDX-Preflight.ps1` now lists the role; sign out/in so the role is reflected; re-run the probe. *(Beta — the exact directory role the Graph endpoint honors is not named on the reference page; verify against current Microsoft documentation.)*

Branch F — 403/404 because the endpoint is not available in your cloud

Symptom: Scope, consent, role, and auth are all clean (A–E pass), you are certain the path is correct, yet the call fails with `403` or `404` — **and you are in a US Gov (GCC High / DoD) or 21Vianet tenant**.

Owner gate: Cloud availability — the feature is **not offered** in your national cloud.

Confirm:

1. `CPADMDX-Preflight.ps1` shows `Environment = USGov` / `USGovDoD` (or you connected to `graph.microsoft.us` / `dod-graph.microsoft.us`).

1. The endpoint's national-cloud table shows ■ for US Gov L4, US Gov L5 (DoD), and 21Vianet — the Copilot admin package and policy-settings endpoints are **Global-only** today.

Fix / workaround: There is no code fix — the endpoint does not exist in your cloud. Use the supported alternatives that **are** offered in your cloud (see `05-testing.md` and Branch J):

- the **Microsoft 365 admin center** → **Agents** surface and its manual/scheduled export;
- any **GA (`/v1.0`) endpoint** whose national-cloud table includes your cloud;
- **Microsoft Purview audit logs** for Copilot/agent activity.

Re-validate: Confirm the chosen fallback returns data in your cloud (testing gate T4). *(Beta — national-cloud availability can expand; re-verify the endpoint's cloud table against current Microsoft documentation before assuming it is still Gov-unavailable.)*

Branch G — 403 because a license prerequisite is not met (acknowledge, then exclude)

Symptom: A–F are clean, but the call still 403s, and the endpoint's reference page states a **license** prerequisite. For the Package Management API, the **current** docs state: *"Access to the Package Management API requires a Microsoft Agent 365 license."*

Owner gate: License.

Confirm:

1. Microsoft 365 admin center → **Billing** → **Licenses** — check whether the relevant users have the license the endpoint's current reference page requires (**Microsoft 365 Copilot**, and for the package catalog, **Agent 365**).

1. Note the **volatility**: the original announcement said *"no additional license or add-on is required,"* and the reference pages were later changed to require **Agent 365**. Always read the requirement from the **current** page, not memory.

Fix / workaround: Ensure the required license is assigned. **The customer has already ruled out Agent 365 licensing as their blocker** — so in this engagement, Branch G is used only to **confirm and exclude** licensing, then move on to Branch H/J. If a license you do not hold is genuinely required, that is a product limitation for you today (Branch J), not a bug — use the supported fallback.

Re-validate: With licensing confirmed present (or excluded), re-run the probe; if it still fails, proceed to Branch H. *(Beta — verify the current license requirement on the endpoint reference page.)*

Branch H — 404 because the path, namespace, or version is wrong

Symptom: `404 Not Found`. Either `code: notFound` with no matching route, or the resource genuinely does not resolve. You may have copied a path from a blog, an older doc, or an internal tool.

Owner gate: Path / version — the URL does not name a real, current route.

Confirm:

1. Compare your URL **character-for-character** against the current reference page: the segments are

exactly `copilot/admin/catalog/packages` (and `/{id}`) for package management, and `copilot/admin/policySettings/{id}` for policy settings. A pluralization, a missing `admin` segment, or an invented sub-resource all 404.

1. Confirm the **version**: try the same path on both `/v1.0/` and `/beta/`. The package endpoints

exist on **both**; a beta-only path called on `/v1.0` (or a route that has since GA'd and moved) will 404 on the version that does not host it.

1. **Distinguish a real 404 from a flighting 404.** If the path is provably correct and still 404s,

this may actually be **Branch R8/F** (not enabled for your tenant/cloud) rather than a typo.

1. For `policySettings`, a `404` with `code: notFound` / *"The setting '…' is not supported"* means the

setting id is not in the supported list — a valid endpoint with an invalid id, not a wrong path.

Fix / workaround: Use the exact current path and a version that hosts it. If the path you were told to use is **undocumented** (not on any current Microsoft Learn reference page), do **not** build a dependency on it — treat it as not-for-third-party-use and go to Branch J. Never fabricate a response shape for a path Microsoft does not document.

Re-validate: Re-run the probe against the corrected, documented path on the correct version.

Branch I — 429 Too Many Requests (throttling)

Symptom: `429 Too Many Requests`, usually with a `Retry-After` response header (seconds). It may appear only under load, in a loop, or when several admins/scripts hit the same tenant.

Owner gate: Throttle.

Confirm:

1. `CPADMDX-Probe-Endpoint.ps1` reports the `429` and echoes the `Retry-After` value if present.
2. Note that preview endpoints often have **tighter, less predictable** burst limits than GA, and Graph

throttling is dynamic (varies by app, tenant, workload, endpoint, and time).

Fix / workaround: Honor `Retry-After` — wait exactly that many seconds, then retry. If no `Retry-After` is returned, use **exponential backoff** (e.g. 5s, 10s, 20s, capped). Reduce call rate: batch, cache, page with `$top`, and avoid tight retry loops. Do **not** hammer a `429` — it extends the throttle.

Re-validate: After the backoff, the retry should return `200` (or a different status to branch on). Confirm your automation now paces itself (best-practices `07`).

Branch J — the data genuinely is not exposed (product limitation)

Symptom: A–I are all clean — you authenticate as a delegated admin, with the correct scope, consent, role, license, path, version, cloud, and no throttling — and yet the endpoint **does not return the data you need** (it 404s persistently for your tenant, returns an empty/partial payload that never includes the field you want, or there simply is no documented endpoint for the thing you are trying to read).

Owner gate: Product capability — this is not a configuration bug; the beta surface does not (yet) expose what you want, or is not intended for your use.

Confirm:

1. Reproduce the *clean* call in a **known-enabled** reference tenant (or Graph Explorer with a

consented admin). If it returns data there but not for you, you are in a **flighting/cloud** gap (Branch F / R8), not a true limitation — wait or open a support case.

1. If **no** documented Graph endpoint returns the field/relationship you need (search the current

Microsoft Learn reference for it and find nothing), or the only path is **undocumented**, treat it as **not exposed / not supported for third-party use**. Do not reverse-engineer an internal route.

Fix / workaround — the supported exit: stop depending on the beta endpoint and use the supported alternative that delivers the data you actually need:

- **Microsoft 365 admin center** → **Agents (Agent Registry)** — the UI inventory and its **export**; use

this for the agent list/sharing view (and see the sibling *Agent Builder Inventory* product for a scheduled export).

- **GA (`/v1.0`) Graph endpoints** — e.g. `copilot/reports/getMicrosoft365CopilotUsageUserDetail`

(`Reports.Read.All`) for usage/adoption; other GA `copilot` paths as they graduate.

- **Microsoft Purview audit logs** — Copilot and agent activity for compliance/reporting.
- **Microsoft Graph PowerShell** GA cmdlets for whatever adjacent data *is* exposed.

Re-validate: Prove the fallback returns the data you need (testing gate T4), and record in `CPADMDX-Findings.json` that the beta endpoint was excluded as a genuine limitation with the date and the doc reference — so you can re-check when the API graduates.

After any fix — re-validate, then decide your dependency

Do not stop at the gate you fixed. Re-run `CPADMDX-Probe-Endpoint.ps1` and confirm you now get a `200` with the expected object (or a clean, documented non-error like a `200` with `value: null` for an unconfigured policy setting). Then run the gates in `05-testing.md` to prove the call is repeatable and to answer the higher question — **is the data actually exposed?** Finally, apply the GA-vs-beta decision in `07-best-practices.md`: keep beta only as an interim/diagnostic source, prefer the GA or admin-center path for anything you depend on, and wrap any beta dependency so a future change fails safe.

Testing & Validation

These gates prove two things: (1) the previously-blocked Graph Copilot admin call **now succeeds**, and (2) the harder question — whether the **data you need is actually exposed** by the endpoint, or whether you

must use a supported fallback. Run them after applying a fix in `03-implementation.md`. Each gate has an explicit **PASS / FAIL** criterion; record the result in `CPADMDX-Diagnostics\CPADMDX-Findings.json`.

> *Run every test as the **delegated admin** you intend to use in production of the automation, in the **cloud** you actually target. Testing as a Global Administrator in Global and then running as a lesser account in GCC High invalidates the result.*

Gate T1 — The call authenticates and authorizes (200, not 4xx)

Procedure:

1. `.\CPADMDX-Preflight.ps1 -Environment <cloud>` — confirm `AuthType = Delegated`, the required scope

is listed, and an admin role (AI Administrator / Global Administrator) is present.

1. `.\CPADMDX-Probe-Endpoint.ps1 -Environment <cloud> -Path 'copilot/admin/catalog/packages' -ApiVersion beta`

PASS: HTTP **200 OK**. For a policy-setting probe, a **200 with** `value: null` also passes — it is the documented response for an unconfigured setting, not an error.

FAIL: Any `401/403/404/429`. Return to the branch the probe names and fix the **first** failing gate. Do not proceed to T2 until T1 passes.

Gate T2 — The response contains the data you actually need

A 200 is necessary but not sufficient — the endpoint may return an object that **omits the field or relationship** you came for.

Procedure:

1. For the package list: confirm the collection is non-empty and each item carries the properties you

need (`id`, `displayName`, `type`, `publisher`, `isBlocked`, `supportedHosts`, `availableTo`, `deployedTo`, `elementTypes`).

1. For a specific package: `.\CPADMDX-Probe-Endpoint.ps1 -Path 'copilot/admin/catalog/packages/{id}'`

and confirm the detail object includes the fields you require (e.g. `elementDetails`, `allowedUsersAndGroups`).

1. If the field you need is **absent from every documented response**, that is a **product limitation**

(Branch J), not a fix-it-later item.

PASS: The 200 payload contains the specific data your use case needs.

FAIL / LIMITATION: The call succeeds but the needed data is not present in the documented schema → record it as **Branch J** and proceed to T4 (prove the fallback).

Gate T3 — The result is repeatable and correctly scoped to your cloud

Procedure:

1. Run the probe **three times**; confirm consistent 200s (a flaky mix of 200/403 suggests

intermittent flighting or throttling, not a stable capability).

1. Confirm `(Get-MgContext).Environment` and the script's printed **GraphHost** match your intended

cloud (`graph.microsoft.com`, `graph.microsoft.us`, or `dod-graph.microsoft.us`).

1. In a **Gov** tenant, confirm you are **not** silently succeeding against a Global endpoint with a

mismatched token.

PASS: Three consistent 200s against the correct cloud host.

FAIL: Inconsistent results → treat as flighting/throttling (Branch F/R8/I); do not build a dependency until stable.

Gate T4 — A supported fallback returns the data (proves the exit path)

Whether or not the beta call succeeded, prove that a **supported** path can deliver the data — this is your production-safe option and your Gov-cloud option.

Procedure:

1. `.\CPADMDX-Test-Fallbacks.ps1 -Environment <cloud>`.

2. Confirm at least one supported path returns the data you need:

• **GA usage reports** (`/v1.0/copilot/reports/getMicrosoft365CopilotUsageUserDetail`) returns rows

for adoption/usage; **or**

• **GA package catalog** (`/v1.0/copilot/admin/catalog/packages`) returns the inventory where

offered; **or**

• the **Microsoft 365 admin center** → **Agents** export / **Purview audit** path yields the equivalent

data (manual confirmation).

PASS: A supported fallback demonstrably returns the needed data in your cloud.

FAIL: No supported path returns it → the capability genuinely does not exist for you yet; document it, cite the source page and date, and set a re-check reminder for when the API graduates.

Gate T5 — Evidence is captured

Procedure: `.\CPADMDX-Report-Findings.ps1` and confirm `CPADMDX-Diagnostics\CPADMDX-Findings.json` and `CPADMDX-Findings.txt` exist and record the failing call, the resolved branch, and the recommended action.

PASS: Both findings files exist and name a resolved branch.

FAIL: Re-run the diagnostic scripts so their JSON outputs exist, then re-run the report.

Exit criteria

You are done when **T1 passes** (the call works) **and** either **T2 passes** (the beta endpoint gives you the data) **or T4 passes** (a supported fallback does), **T3** confirms stability in the correct cloud, and **T5** has captured the evidence. If only T4 passes, your production path is the fallback — apply the dependency guidance in `07-best-practices.md`.

Troubleshooting — edge cases and gotchas

The **Diagnostic Runbook** (`03-implementation.md`) is the main symptom tree for a blocked call. This file covers the harder edges: getting the **tooling and consent** to cooperate, and the three preview-specific traps — **throttling storms**, **tenant-not-enabled flighting**, and **undocumented-endpoint deprecation** — that masquerade as ordinary errors.

> *Reminder: this is a **beta** surface. Any behavior below can change without notice; re-verify against current Microsoft documentation before treating a workaround as permanent.*

Tooling, sign-in, and consent

`Connect-MgGraph` **fails or the module is missing.**

- Install: `Install-Module Microsoft.Graph -Scope CurrentUser` (and `Microsoft.Graph.Beta` for beta

cmdlet coverage). Confirm with `Get-Module Microsoft.Graph -ListAvailable`.

- An **old SDK** can lack newer environments or cmdlets — `Update-Module Microsoft.Graph`.

The consent prompt says an administrator must approve.

- The Copilot admin scopes require **admin consent**. Have a Global Administrator grant it once for

Microsoft Graph Command Line Tools (appId `14d82eec-204b-4c2f-b7e8-296a70dad67c`) or your custom app. See Runbook **Branch D**.

You consented, but the scope is still not in the token.

- You may have consented in a **different cloud** or **different tenant**. Confirm

`(Get-MgContext).Environment` and `.TenantId`. Reconnect explicitly: `Connect-MgGraph -Environment <cloud> -TenantId <id> -Scopes ...`.

You are sure you have the permission but still get 403.

- Nine times out of ten this is a **role** gap, not a scope gap (Runbook **Branch E**), or an

app-only token where delegated is required (**Branch B**). Run `CPADMDX-Resolve-Scopes.ps1` — it separates the two.

`jwt.ms` **shows the scope but the call still 403s.**

• The scope is present but the **user lacks the admin role**, or the **feature is not flighted** to your tenant, or you are in a **Gov cloud** where the endpoint does not exist. Work Branches E → F → R8 in that order.

Throttling storms (429)

Every call suddenly 429s, not just heavy ones.

• You (or another script/admin on the same tenant) tripped a **dynamic** throttle. Graph throttling varies by app, tenant, workload, endpoint, and time of day; a burst earlier can throttle you now.

• **Honor** `Retry-After` exactly. If absent, back off exponentially (5s→10s→20s→…, capped). Do **not** retry in a tight loop — it extends the penalty window.

A batch job throttles where single calls do not.

• Reduce concurrency and add pacing between requests; page with `$top`/`$filter` instead of pulling everything; cache results you will reuse. Preview endpoints have **less** burst headroom than GA — size the job accordingly.

429s only in one cloud or one tenant.

• Throttle budgets are per-tenant/per-app. A shared automation account across many tenants amplifies throttling — give each tenant its own pacing.

Tenant-not-enabled (flighting) — the 403/404 that is not your fault

The call is provably correct (path, version, scope, role, consent, cloud) and still 403s or 404s.

• The preview feature has **not rolled out to your tenant**. This is Microsoft-side flighting; there is **no self-service switch**. Multiple tenants report exactly this on the package-catalog endpoint.

• **Confirm** by reproducing the same call in a tenant known to be enabled (or Graph Explorer with a consented admin). Success there + failure for you = flighting, not misconfiguration.

• **Cross-check the UI:** if the Microsoft 365 admin center **Agents / Agent Registry** surface is absent in your tenant, the mirroring API is likely not flighted either.

• **Action:** wait for the rollout, or open a Microsoft support case for eligibility; meanwhile use the supported fallbacks (Runbook Branch F/J). Record the `request-id` and date in `CPADMDX-Findings.json`.

A 404 that is really "not enabled."

• A `404` is not always a typo. If the path matches the current reference **character-for-character** on a version that hosts it, and it still 404s, suspect **flighting or cloud unavailability** rather than a wrong route (Runbook Branch H → F/R8).

Undocumented-endpoint deprecation and drift

A path that worked last month now 404s or returns a different shape.

- Beta paths can be **renamed, moved to GA, or removed** without notice. Re-check the current reference

page: the resource may now live on `/v1.0`, or a property may have been renamed/removed.

- Your wrapper should **tolerate missing/renamed fields** and fall back automatically (Best Practices §5). If you hard-coded the shape, that is the bug to fix — not the endpoint.

A blog or internal snippet uses a path you cannot find in Microsoft Learn.

- Treat an **undocumented** route as **not for third-party use**: it can vanish without notice and may violate the API terms. Do **not** build on it and do **not** fabricate its response shape. Use the supported path (GA endpoint, admin-center export, Purview audit) and file a feature request if the capability matters.

The license/permission requirement changed under you.

- The package-catalog endpoint moved from *"no additional license"* to *"requires Microsoft Agent 365,"* and lists application permissions that are rejected at runtime. Always read licensing and permissions from the **current** reference page, not memory or an older doc. (**The customer has ruled out Agent 365 licensing as the blocker** — confirm and exclude via Runbook Branch G, then move on.)

Policy-settings-specific responses (not failures)

If you are probing `copilot/admin/policySettings/{id}`, several non-200 responses are **by design**, not bugs:

- `200` **with** `value: null` **(and/or** `policyId: null`**)** — the setting is not configured / no tenant policy exists. Expected.
- `404` `unsupportedSetting` — the setting **id** is not in the supported list. Fix the id, not the path.
- `422` `groupScopedSettingNotSupported` — the setting is configured at **group** level; this endpoint returns **tenant-level** only.
- `502` `tooManyGroupPolicies` — the tenant exceeds the supported number of group-level policies.

Read these from the error `innerError.code`, not just the status number.

When the method itself stalls

- **You fixed a gate and it still fails.** You probably fixed a **later** gate while an **earlier** one is still failing. Return to Step 0, re-read scopes/roles/auth/cloud in gate order, and fix the **first** failing gate.
- **Results are intermittent across tenants or accounts.** Compare a working tenant/account's

`CPADMDX-Findings.json` against the failing one — the difference (cloud, role, flighting, scope) is your cause.

- **Nothing supported returns the data.** Accept it as a genuine limitation (Runbook Branch J), record the source page and date, and set a re-check for when the API graduates to GA.

Best Practices — when to depend on beta vs wait for GA

The hardest decision this runbook leaves you with is not *"how do I make the call work"* but *"should I build on it."* These practices answer that.

1. Treat `/beta/copilot/admin/*` as interim, not production

Microsoft's own notice on every beta page is unambiguous: *"APIs under the `/beta` version are subject to change. Use of these APIs in production applications is not supported."* That is not boilerplate — the Copilot admin surface has already changed its **license requirement** (from *"no additional license"* to *"requires Microsoft Agent 365"*) and its **documented-vs-enforced permission model** (application permissions listed but rejected at runtime) during preview. Build on beta only for **diagnostics, one-off pulls, or interim automation you can afford to have break**.

2. Prefer GA (`/v1.0`) or the admin center for anything you depend on

Before you wire a beta call into a scheduled job, check for a supported equivalent:

- a **GA (`/v1.0`) path** for the same resource (the package catalog and usage reports both have GA paths);
- the **Microsoft 365 admin center** → **Agents** export;
- **Microsoft Purview** audit data.

If a GA or admin-center path delivers the data, use it — even if the beta call is momentarily more convenient.

3. Use delegated auth with a least-privilege admin role

These endpoints require a **signed-in user** (application-only tokens are rejected). Sign in as an account holding **AI Administrator** — the least-privilege role for Copilot/agent administration — rather than Global Administrator. Reserve Global Administrator for genuine tenant-wide changes, and use **PIM** to activate the role only when running the diagnostic.

4. Request only the scopes you need, and name them exactly

Ask for `CopilotPackages.Read.All` (read) rather than `ReadWrite` unless you must write; use `CopilotPolicySettings.Read` for policy reads. Never request `ReadWrite`/write scopes "just in case" — they widen consent and audit exposure. Get the scope names character-exact from the current reference.

5. Wrap every beta dependency so it fails safe

If you must depend on a beta call, isolate it behind one function that:

- checks the **HTTP status** and never assumes a shape — parse defensively, tolerate new/removed fields;
- treats a `404`/`403` as *"capability unavailable"* and falls back to the supported path automatically;
- logs the `request-id` on every failure for support cases;
- carries a **doc-reference + last-verified date** so you know when to re-check.

A single wrapper means a Microsoft-side change touches one place, not your whole codebase.

6. Pace your calls and honor throttling

Cache results, page with `$top`/`$filter`, and avoid tight retry loops. On `429`, **honor** `Retry-After`; if absent, use exponential backoff. Preview endpoints have tighter, less predictable burst capacity than GA — assume less headroom, not more.

7. Plan for Government and sovereign clouds explicitly

The Copilot admin beta endpoints are **Global-only** today (not GCC High / DoD / 21Vianet). If any of your tenants are sovereign, design the **fallback as the primary** path there from day one — do not assume parity, and re-verify the per-endpoint national-cloud table before each release. *(Beta — cloud availability can expand; verify against current Microsoft documentation.)*

8. Re-verify on a schedule, and record what you excluded

Preview behavior drifts. Keep a short register of every Copilot admin endpoint you use, its version, its scope/role/license requirement, its cloud availability, and the **date you last verified** it against Microsoft Learn. When you exclude an endpoint as a genuine limitation (Branch J), record the source and date so you re-check when it graduates to GA.

9. Keep the diagnostic evidence

Retain `CPADMDX-Findings.json` from each investigation. It is your fastest way to reopen a Microsoft support case with the exact `request-id`, and your record of what the endpoint did (and did not) return on a given date — invaluable when the beta shape changes underneath you.

10. Don't reverse-engineer undocumented routes

If the only path that returns what you want is **not on any current Microsoft Learn reference page**, treat it as **not for third-party use** and do not build on it. An undocumented route can be removed without notice and may violate the API terms of use. Use the supported fallback and, if the capability matters to you, file the feature request.

Deliverables Summary

Everything in this solution, and how the pieces fit together. Delivered as a single ZIP you keep and reuse across unlimited investigations.

Documents (Word + PDF)

Document	What it gives you
Executive Summary	The problem, the method, and the key verified facts for a decision-maker.
Solution Overview (`01`)	What the `/beta/copilot/admin/*` surface is, the GA-vs-beta support posture, the 401-vs-403 and scope-vs-role distinctions, and the diagnostic model.
Requirements (`02`)	The nine prerequisites to call the endpoints — tooling, delegated admin, exact scopes, admin role, consent, correct cloud, licensing, tenant flighting, and the evidence file — each with Description / Why / Verify / Implement-if-missing.
Global Requirements — Copilot	The category-wide Copilot prerequisites (tenant, licensing, identity, data readiness, roles, governance).
Diagnostic Runbook (`03`)	The core: Step 0 fact-finding plus **Branches A–J**, one per root cause, each with symptom → confirm → fix/workaround → re-validate.
Testing & Validation (`05`)	Five PASS/FAIL gates that prove the call now works **and** whether the data is truly exposed or needs a fallback.
Troubleshooting (`06`)	Edge cases: throttling storms, tenant-not-enabled flighting, undocumented-endpoint deprecation, tooling/consent snags.
Best Practices (`07`)	When to depend on beta vs wait for GA, and how to wrap a beta dependency so it fails safe.

PowerShell scripts (5, read-only, cloud-aware)

All scripts accept `-Environment Global | USGov | USGovDoD` and perform **GET requests only** — they never change tenant state and never fabricate a response for a beta endpoint.

Script	Purpose
`CPADMDX-Preflight.ps1`	Report the signed-in account, **delegated vs app-only**, token **scopes**, **directory roles**, and cloud — Step 0.
`CPADMDX-Probe-Endpoint.ps1`	Call the target endpoint and **decode** the 401/403/404/429 into the likely branch, printing the real error `code`/`message`/`request-id`.
`CPADMDX-Resolve-Scopes.ps1`	Report **have vs need** for scopes and admin role, isolating a scope gap (C/D) from a role gap (E).

Script	Purpose
`CPADMDX-Test-Fallbacks.ps1`	Exercise the **supported alternatives** (GA usage reports, GA package catalog, admin-center/Purview pointers) to prove the exit path.
`CPADMDX-Report-Findings.ps1`	Consolidate all outputs into `CPADMDX-Diagnostics\CPADMDX-Findings.json` + `.txt` with the resolved branch and next action.

Named artifacts you create

- `CPADMDX-Diagnostics` — the working folder for all diagnostic output.
- `CPADMDX-Findings.json` / `CPADMDX-Findings.txt` — the consolidated evidence file (support-case ready).

What this solution deliberately does not include

- It does **not** build or configure Copilot agents, and does **not** reproduce the **agent inventory + sharing export** deliverable — that is the sibling *Copilot Agent Builder — Inventory, Sharing, Governance & Export* product. This runbook unblocks the API and points you there for the export.
- It does **not** invent undocumented endpoints or response shapes; where data is not exposed, it names the supported fallback instead.

How to use it

Start at the **Diagnostic Runbook** Step 0, run `CPADMDX-Preflight.ps1` and `CPADMDX-Probe-Endpoint.ps1`, follow the branch the probe names, apply the fix, then confirm with the **Testing** gates. Keep the findings file for your records and for Microsoft support.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CPADMDX-Preflight.ps1

Purpose: Step 0 of the diagnostic runbook. Establishes the facts you must know before branching: - who you are signed in as, and whether the token is DELEGATED or APP-ONLY (Branch B), - which permission scopes are actually in the token (Branch C/D), - which Entra directory roles the signed-in user holds - e.g. AI Administrator / Global Administrator (Branch E), - which cloud (Global / USGov / USGovDoD) the token targets (Branch A/F). It changes nothing in the tenant.

How to run: 1. Install the Graph SDK if needed: Install-Module Microsoft.Graph -Scope CurrentUser 2. Run, choosing your cloud: .\CPADMDX-Preflight.ps1 -Environment Global .\CPADMDX-Preflight.ps1 -Environment USGov # GCC High (graph.microsoft.us) .\CPADMDX-Preflight.ps1 -Environment USGovDoD # DoD (dod-graph.microsoft.us) 3. Sign in interactively as the admin you intend to use. Consent to the requested read scopes.

CPADMDX-Probe-Endpoint.ps1

Purpose: Reproduces the failing call with the REAL status code and error body visible (the console UI often hides it), then prints the most likely runbook branch. It NEVER fabricates a response shape - on failure it reports exactly what Graph returned.

How to run: 1. Connect first (or let this script connect): see CPADMDX-Preflight.ps1 2. Probe the default package-catalog endpoint: .\CPADMDX-Probe-Endpoint.ps1 -Environment Global 3. Probe a specific package by id, or any other admin path/version: .\CPADMDX-Probe-Endpoint.ps1 -Path 'copilot/admin/catalog/packages/{id}' -ApiVersion beta .\CPADMDX-Probe-Endpoint.ps1 -Path 'copilot/admin/policySettings/microsoft.copilot.allowwebsearch' .\CPADMDX-Probe-Endpoint.ps1 -Path 'copilot/admin/catalog/packages' -ApiVersion v1.0

CPADMDX-Report-Findings.ps1

Purpose: Aggregates CPADMDX-Preflight.json, CPADMDX-Probe.json, CPADMDX-Scopes.json, and CPADMDX-Fallbacks.json (whichever exist) into one artifact you can keep or attach to a Microsoft support case. It derives the single most likely branch from the collected facts. It creates the CPADMDX-Diagnostics folder if missing and changes nothing in the tenant.

How to run: 1. Run the other scripts first (Preflight, Probe, Resolve-Scopes, optionally Test-Fallbacks). 2. Run: .\CPADMDX-Report-Findings.ps1 .\CPADMDX-Report-Findings.ps1 -OutputFolder .\CPADMDX-Diagnostics

CPADMDX-Resolve-Scopes.ps1

Purpose: Answers the most common confusion: "I have the permission and still get 403." It prints a have-vs-need table for the endpoint family you name, using the verified scope requirements, and shows whether the signed-in user holds an admin role. It does not change anything.

How to run: 1. Connect first (or let this script connect) - see CPADMDX-Preflight.ps1. 2. Name the endpoint family you are calling: .\CPADMDX-Resolve-Scopes.ps1 -Endpoint Packages .\CPADMDX-Resolve-Scopes.ps1 -Endpoint PolicySettings .\CPADMDX-Resolve-Scopes.ps1 -Endpoint

UsageReports .\CPADMDX-Resolve-Scopes.ps1 -Endpoint PackagesWrite

CPADMDX-Test-Fallbacks.ps1

Purpose: When a /beta/copilot/admin/* call cannot be relied on (Branch F/J), this script exercises the supported fallbacks and reports which ones return data in YOUR tenant/cloud: 1. GA usage reports: GET /v1.0/copilot/reports/getMicrosoft365CopilotUsageUserDetail(period='D7') 2. GA package catalog: GET /v1.0/copilot/admin/catalog/packages (if offered in your cloud) 3. Prints the non-API fallbacks (M365 admin center Agents export; Purview audit) to consider. It performs GET requests only and never fabricates a response.

How to run: 1. Connect with the fallback scopes: Connect-MgGraph -Environment Global -Scopes 'Reports.Read.All','CopilotPackages.Read.All' 2. Run: .\CPADMDX-Test-Fallbacks.ps1 -Environment Global .\CPADMDX-Test-Fallbacks.ps1 -Environment USGov # confirm what IS available in GCC High